

SPOTLIGHT: Scalable and Cost-Efficient Federated Learning on Spot Instances

Anonymous Authors
Paper: #XXX

Abstract

1 Introduction

Hybrid spot/on-demand strategies [10, 29] have been extensively studied.

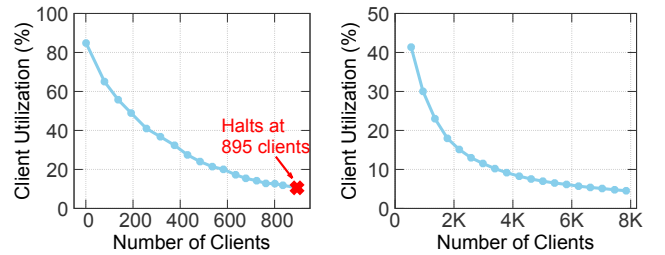
2 Motivation

FL trains models without centralizing data, and its deployments have grown accordingly. Production systems already serve tens of millions of devices [3]. Each device returns a model update that must be merged into the global model. The aggregation layer must handle all of this work. Existing FL frameworks, including Flower [2], IBM FL [16], FATE [15], and Papaya [7], implement this layer as a single aggregator that manages the global model. We examine two challenges, the scalability limits of a single aggregator (§2.1) and the infrastructure cost of scaling the aggregation tier (§2.2), and show that their solutions cannot be designed in isolation (§2.3).

2.1 Scaling FL aggregation

In FL, an aggregator is responsible for three operations [3, 7]. It manages the lifecycle of connecting clients, selects and forwards devices into training rounds, and incorporates the returned updates into the global model [9]. Deployments now reach tens of millions of devices [3]. Building an aggregation layer for that scale places significant load on the aggregator, as the demand of all three operations rises with the number of clients.

Aggregators incorporate client updates either synchronously or asynchronously. Flower [2] adopts synchronous client updates, requiring a quorum of clients before each round. On the other hand, Papaya [7] uses asynchronous client updates, applying each update on arrival without waiting for a quorum. To see how each behaves as the client population increases, we measure client utilization. Fig. 1 shows the result



(a) Flower, 100 to 895 clients. (b) Papaya, 500 to 8,000 clients.

Figure 1: **Client utilization degrades as the client population grows in both Flower and Papaya FL systems.**

with a 10MB payload. Flower’s utilization (Fig. 1a) decreases from 85% to below 12%, and at 895 clients it halts entirely. Papaya sustains a much larger population, but its utilization (Fig. 1b) decreases from 41% to near zero by 8,000 clients. The two systems fail differently. Flower halts, while Papaya keeps running but no longer benefits from additional clients. We could confirm that both systems reach a threshold at which a single aggregator can no longer process the load.

An intuitive approach to address this challenge is to add more aggregators. In a stateless service, this approach can be sufficient, as any instance can serve any request. However, FL aggregation is *stateful*: each aggregator maintains a running model that reflects only the updates it has received. Therefore, partitioning clients across instances also partitions the model. Each instance produces a sub-global model that is accurate on its own subset of clients but incomplete for the rest. Adding or replicating the aggregator alone is not enough, as it can yield several partial models and no global one. Creating and maintaining a single global model requires an explicit step that merges the sub-global models.

The merging step can be performed within the aggregator layer, with the aggregators combining their sub-global models. But this approach requires every aggregator to coordinate with every other, and the coordination overhead increases with the number of aggregators. Placing the merge in a separate tier

removes this overhead, as each tier then coordinates only within itself. Each lower-tier aggregator handles a subset of clients, so its load fluctuates with client participation. The upper-tier aggregator merges the sub-global models, so its load depends on the number of lower-tier aggregators rather than on the client count.

Furthermore, because these loads change throughout FL operation, the number of aggregators in each tier cannot be fixed in advance. The system, therefore, should provision and deprovision aggregators as the load changes. When and how to perform these operations should be carefully designed, in addition to introducing the merging tier.

Design Insight #1: Scaling FL aggregation requires a two-tier hierarchy, where a lower tier aggregates client updates and an upper tier merges the resulting sub-global models. Because load varies throughout FL operation, this hierarchy must also be elastic, and deciding when and how to scale it is an additional design challenge.

2.2 The cost of FL running at scale

Even after the scalability challenge is addressed, another challenge remains. In practice, the aggregation layer runs on always-on, on-demand cloud VMs [3, 21, 26], and scaling it to serve large client populations makes the cloud cost prohibitive. To quantify this cost, we measure the per-round aggregator time for training ResNet-18 on CIFAR-100 with FedAvg [17] on an AWS p3.2xlarge instance, and the median round takes 732 seconds. Prior work reports that this task converges in 1,198 rounds [25], so a full run requires roughly 244.5 hours of aggregator time. At the on-demand rate of \$3.06 per hour, this amounts to \$748.25 just to train a single model to convergence.

Spot instances offer a much cheaper alternative, typically 3 – 10 $\times$ cheaper than on-demand [8, 24, 28]. The caveat is that the provider can reclaim a spot instance at any time, causing the aggregator to lose its progress. To estimate the potential cost advantage, we first consider the ideal case in which no preemption occurs. In that case, the same 244.5-hour run at the spot rate of \$0.345 per hour [4, 13] will cost only \$84.36, an 88.7% reduction over on-demand.

In practice, however, spot preemptions do occur. Without proper fault-tolerance mechanisms, a preemption reclaims the spot VM, causing the aggregator to lose all progress. To continue the FL process, a new instance must be provisioned and the computation restarted from scratch. We simulate spot preemptions on the same training task, using a real availability trace [28] and restarting on every preemption. Averaged over ten runs, the aggregator completes only 156 of the 1,198 rounds within the 244.5-hour ideal window, so finishing the full run would take roughly 1,875 hours on spot instances. At \$0.345 per hour, this comes to \$646.88, just 13.5% below the on-demand cost. Fig. 2 compares the three cases. Even if we

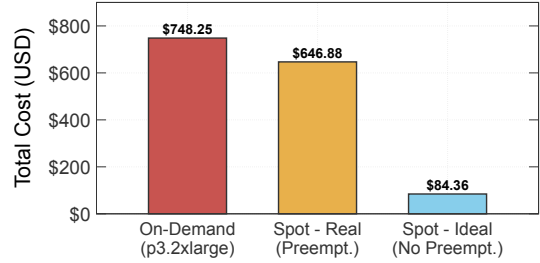


Figure 2: **Aggregator cost to train ResNet-18 on CIFAR-100 to convergence on a p3.2xlarge instance (1,198 rounds [25]).** Without fault tolerance, preemptions force full restarts, raising the spot cost from an ideal \$84.36 to \$646.88, close to the on-demand cost of \$748.25.

disregard the effect of preemptions on model accuracy, spot instances without fault tolerance offer only a negligible cost advantage over on-demand.

Recovering the cost advantage of spot instances therefore requires fault tolerance. Periodic checkpointing saves the model so that a preempted run can resume from the last checkpoint [xxx₁, (IKK₁: chkpt papers)], but determining an optimal checkpoint interval is non-trivial and adds implementation complexity. Aggregator relocation moves the aggregator to another instance before the current spot VM is reclaimed [xxx₂], but provisioning a new instance takes time and delays aggregation. Providers send a termination signal shortly before reclaiming an instance (e.g., 2 minutes [1] or 30 seconds [18]), but the termination notice is short, only two minutes on AWS and thirty seconds on GCP. These techniques have not been applied to or optimized for FL aggregation on spot instances. Therefore, we address fault tolerance for FL on spot not only by adapting them to the setting, but also by developing new mechanisms designed for FL aggregation.

Design Insight #2: The cost advantage of FL aggregation on spot instances is achievable only with fault tolerance. This fault tolerance requires both applying well-established principles and developing new techniques tailored to FL aggregation.

2.3 Coupling FL scaling and fault tolerance

We discussed two FL challenges and their corresponding solutions in §2.1 and §2.2. The scalability challenge motivates us to employ autoscaling, which elastically provisions and deprovisions aggregators to handle client load fluctuations. The cost of running at scale motivates us to deploy the FL system on spot instances, which in turn require fault tolerance to address spot preemptions. However, these two mechanisms cannot be developed in isolation. Both act on the same resources and the same set of aggregators currently running.

Consider the following scenario when these two mecha-

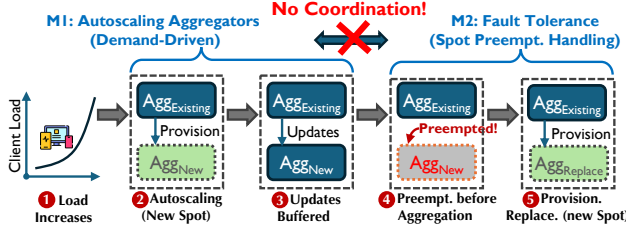


Figure 3: **A conflict between autoscaling (M1) and fault tolerance (M2) when the two operate independently.** Autoscaling provisions a new aggregator on spot to handle rising load (P1–P3), but the instance is preempted before its buffered updates are aggregated (P4), and a replacement must be provisioned (P5). Because the two mechanisms act on the same aggregators without coordination, autoscaling places load on an instance that fault tolerance must immediately recover.

nisms operate independently, as illustrated in Fig. 3. As client load rises (1), autoscaling provisions a new aggregator to handle the additional client updates (2). This aggregator runs on a spot instance, and its remaining lifetime is unknown at the time of provisioning. Once the new aggregator is ready, incoming client updates are placed on it and accumulate in its buffer before aggregation (3). Unfortunately, if this instance is preempted before these buffered updates are aggregated (4), they are lost along with the client computation they carry, and a replacement must be provisioned to continue (5).

The root cause of this problem is that the two mechanisms are designed for different goals over the same aggregators and resources. Autoscaling (M1, §2.1) reacts to load fluctuation, while fault tolerance (M2, §2.2) handles preemption. Because the two operate without coordination, a decision deemed beneficial to one can be detrimental to the other, as the scenario above shows. Therefore, we argue that scaling and fault tolerance must be coordinated by a single integrated policy rather than run as two operations side by side. SPOTLIGHT builds on this insight, which we describe in the next section.

Design Insight #3: Because scaling and fault tolerance operate on the same aggregators, controlling them independently may result in conflicting decisions. Achieving both scalability and cost efficiency requires coordinating them under a single integrated policy.

3 System Overview

This section presents SPOTLIGHT’s architecture, which addresses these three challenges. Fig. 4 shows its main components. Aggregation runs in an *aggregator pool*, where local and global aggregators process client updates and produce the global model. A *resource manager* (RM) controls this pool, decides how many aggregators to run, where to place them,

and when to move them. A *scheduler* routes incoming client updates to aggregators in the pool. A *performance manager* tracks aggregator throughput and computes the threshold that drives scaling, and a persistent store holds the model state. To reduce cost, SPOTLIGHT runs aggregators on spot instances, which can be preempted at any time. The controller side of SPOTLIGHT, including resource manager, scheduler, performance manager, and persistent store, runs on on-demand instances and are therefore not subject to preemption.

Two-tiered aggregation. The aggregation pool is organized into two tiers. A *local aggregator* (§4.2 xxx₃) handles a subset of clients and produces a sub-global model, and a *global aggregator* (§4.2 xxx₄) merges these sub-global models into a single global model. This two-tier structure limits each tier’s coordination scope. A local aggregator coordinates only with its assigned clients, and the global aggregator with only a fixed number of sub-global models. To support scalability a *scaler* (§4.2 xxx₅) in the resource manager provisions and de-provisions aggregators in each tier independently, as client load changes.

Proactive fault tolerance. As aggregators run on spot instances, any aggregator can be preempted at any time [xxx₆ (IKK₂: spotlake, skypilot)]. When an instance is reclaimed, the updates it has buffered but not yet aggregated are lost, and provisioning a replacement takes time, so aggregation stalls while a new instance starts up [6]. SPOTLIGHT instead preserves aggregation state ahead of a preemption. A *migrator* (§4.3 xxx₇) estimates each aggregator’s likelihood of preemption from its instance’s uptime, and when an aggregator is likely to be preempted soon, it migrates the workload and checkpoints the state before the instance is lost. If the instance is reclaimed before the move completes, the aggregator resumes from its most recent checkpoint. If no spot instance is available to replace it, the workload moves instead to a *failover edge aggregator* (§4.3 xxx₈), which transfers its model back to the pool once the spot instance returns.

Coordinated scaling and fault tolerance. Both scalability and fault tolerance mechanisms operate on the same aggregators. The scaler provisions and deprovisions aggregators as load changes, and the migrator moves workloads to another aggregator when one is at risk of preemption. When these two decide independently, their operations can conflict, as shown in §2.3. SPOTLIGHT therefore integrates both decisions in the RM, which monitors the load on every aggregator and the preemption risk of every instance. Scaling decisions consider preemption risk, and migration decisions consider current load, ensuring that the two operations do not conflict.

Execution flow. Fig. 4 shows how these components work together. Client updates arrive through the gateway, and the scheduler assigns each to a local aggregator in the pool (1). Each local aggregator buffers the updates it receives and produces a sub-global model, which the global aggregator merges

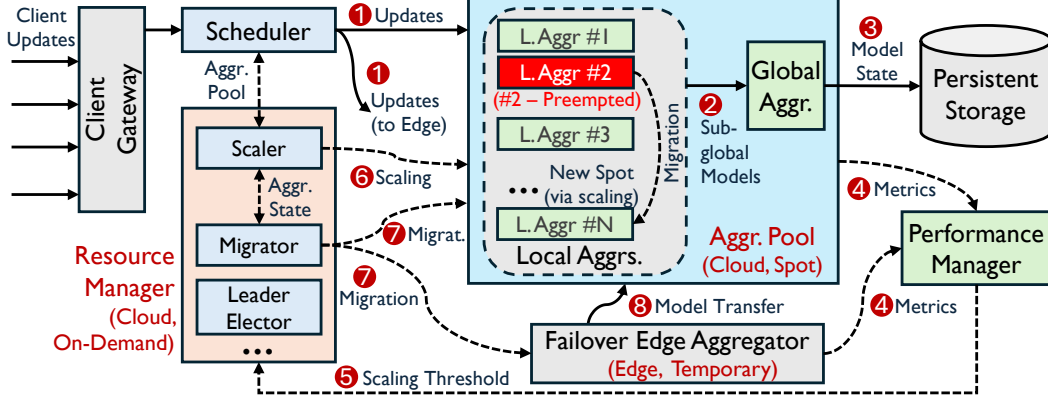


Figure 4: **SPOTLIGHT architecture.** Aggregation runs on spot instances in two tiers, while the resource manager that controls it runs on on-demand instances (which is never preempted). The scheduler routes client updates to aggregators drawn from the pool the resource manager maintains, and within the resource manager the scaler and migrator make scaling and migration decisions together over shared aggregator state. Solid arrows carry data and dashed arrows carry control, and numbered steps are described in the text. L.Aggr.: Local aggregators, Global Aggr.: Global aggregators.

into the global model (2) and writes to the persistent store (3). While this proceeds, aggregators report their throughput to the performance manager (4), which computes a scaling threshold for the resource manager (5). The scaler acts on that threshold, provisioning or removing aggregators as load shifts (6). The migrator meanwhile monitors for preemption risk and moves its workloads to a different aggregator before its instance is preempted (7), falling back to the failover edge aggregator when a preemption arrives first and taking the model back once a replacement is ready (8).

4 System Components

Building on the three design principles established in §??, this section describes how SPOTLIGHT realizes them in practice. The aggregation tiers realize two-tiered aggregation through independently scalable Sub-Global and Global Aggregators. The Resource Manager handles scheduling, orchestrates scaling across both tiers, and migrates workloads before preemption occurs, preserving training continuity and the cost efficiency of spot infrastructure. We begin with the update lifecycle (§4.1), which establishes the end-to-end path a client update follows through the system, then describe each component in detail.

4.1 Update Lifecycle

In SPOTLIGHT, client updates flow through two aggregation tiers, accumulating into sub-global models before being merged into the global model. Clients maintain persistent gRPC connections to Envoy, which routes them to the Resource Manager. The RM forwards client updates to the appropriate Sub-Global Aggregator asynchronously in the background, decoupling client-facing latency from aggregator throughput. The Sub-Global Aggregator accumulates updates in an in-memory buffer and triggers aggregation once the floor threshold is met. The resulting sub-global model is pushed to the Global Aggregator, which incorporates it into the global model. Sub-Global Aggregators pull the updated global model and propagate it down the hierarchy to clients.

4.2 Two-Tiered Aggregation

SPOTLIGHT distributes aggregation across two tiers, allowing each tier to scale independently as client participation grows. **Aggregators** operate at two distinct points in the hierarchy with different responsibilities. Sub-Global Aggregators receive updates from the RM, aggregate them into a sub-global model, and ensure clients receive the latest available model. Global Aggregators receive sub-global models from all Sub-Global Aggregators and merge them into a single global model. Both tiers adopt FedBuff’s [19] buffered aggregation model, but use the buffer as a floor rather than a strict barrier, triggering aggregation as soon as the minimum threshold is met, avoiding stalls from incomplete batches and improving aggregator throughput. Sub-Global Aggregator workload scales directly with client participation and can vary significantly as clients join or leave training, while Global Aggregator workload is bounded by the number of Sub-Global Aggregators, making it more stable. This difference in workload characteristics motivates distinct scaling policies at each tier.

Scaler elastically scales both aggregation tiers in response to client load and system performance. The scaling of Sub-Global and Global Aggregators is managed by the RM. The

scaling policy used by SPOTLIGHT for Sub-Global Aggregators utilizes client participation and utilization to scale horizontally, while Global Aggregators use memory utilization to scale. The difference in scaling signal between the two tiers is motivated by the workload characteristics of each tier. The algorithm used for scaling Sub-Global Aggregators is described in Algorithm 2 (Figure 7). The Scaler utilizes this algorithm along with real-time measurements and scaling parameters obtained from the Performance Manager to scale Sub-Global Aggregators. For Global Aggregators, SPOTLIGHT internally implements the Kubernetes HPA algorithm, with scaling parameters likewise supplied by the Performance Manager.

Performance Manager dynamically adjusts the scaling parameters consumed by the Scaler across both aggregation tiers in response to observed system performance. The Performance Manager implements a Proportional-Integral-Derivative (PID) controller that monitors aggregation throughput and reacts to changes between consecutive control cycles. A fixed setpoint is avoided as FL workloads operate under variable client participation, making a stable throughput baseline difficult to establish without extensive tuning. Instead, the controller reacts to changes in throughput between cycles, tightening scaling parameters when throughput degrades and relaxing them when it improves. The error signal $e(t)$ and controller output $u(t)$ are defined in Equations 1 and 2. The PID parameters used in our deployment are listed in Table 1.

Scheduler places incoming client updates onto available Sub-Global Aggregators using a queue-aware placement policy, formally defined in Algorithm-1 (Figure 6) and Figure 5. In SPOTLIGHT, aggregation begins once the minimum number of client updates are present on the aggregator. The bursty nature of FL workloads means naive placement policies risk stalling aggregation by spreading updates thinly across aggregators, none of which reach the floor threshold. The Scheduler addresses this by selecting the two aggregators with the largest queues from the active pool, concentrating updates to trigger aggregation as early as possible. The selected aggregator’s AFS is incremented on every scheduling decision and decays over time since the last selection. When aggregators are equally ranked by queue length, AFS serves as a tiebreaker, deprioritizing aggregators with more recent scheduling decisions in favor of those that have been selected less recently. The decision flow is illustrated in Figure 5.

Version Consistency As Sub-Global and Global Aggregators scale independently, concurrent aggregation events across tiers can produce divergent model versions, causing clients to train on stale or inconsistent state. SPOTLIGHT addresses this with a shared monotonic version counter in MongoDB, atomically incremented by all aggregators whenever a new aggregated model is produced. The version counter provides a consistent ordering of aggregation events across the hierarchy, ensuring that clients always receive the most recent model regardless of which aggregator serves it. All aggregators detect and recover stale in-memory state after restarts by comparing

Table 1: PID Controller Values

Parameter	Value
K_p	0.05
K_i	0.02
K_d	0.10
γ	0.97
I_{max}	10.0

their local version against the shared counter, pulling the latest model from persistent storage when a discrepancy is found.

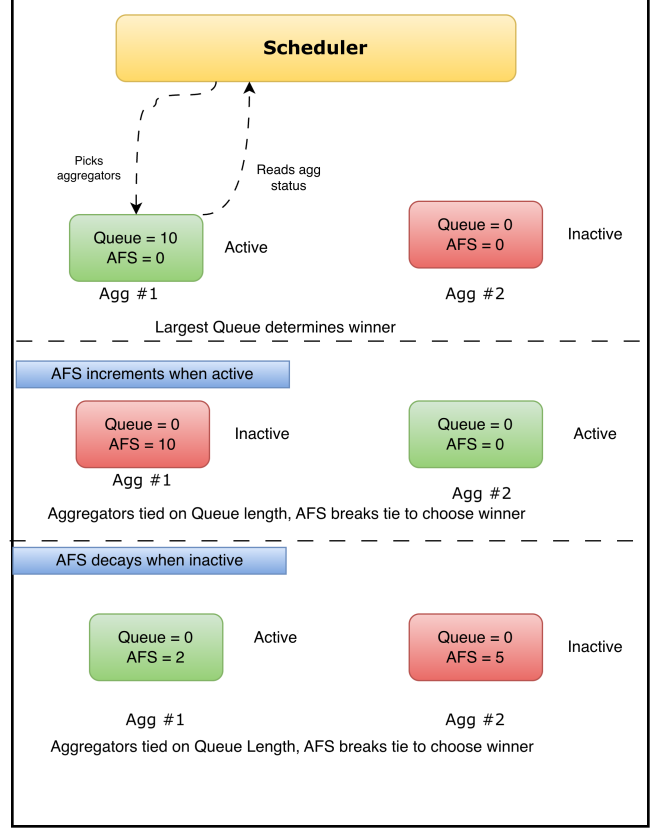


Figure 5: **Scheduling algorithm.** SPOTLIGHT primarily selects the aggregator with the largest queue to begin aggregation; when no aggregator has a clear lead, the Aggregation Fairness Score (AFS) breaks the tie. AFS measures how recently an aggregator was selected, with a higher score indicating heavier recent use. To avoid overloading any single aggregator, the scheduler favors aggregators that have not been picked recently, incrementing AFS each time an aggregator is chosen and decaying it exponentially over time.

$$e(t) = T_{\text{current}} - T_{\text{previous}} \quad (1)$$

$$u(t) = K_p e(t) + K_i \cdot \text{clamp}(\gamma \cdot I_{t-1} + e(t) \cdot \Delta t, -I_{max}, I_{max}) + K_d \frac{de(t)}{dt} \quad (2)$$

Algorithm 1 Scheduling Algorithm

Input: M : metrics $\rightarrow$ aggregator $\mapsto \langle q \rangle$, where q = queue size
Input: AFS: fairness score map (as of last update)
Input: AFS_{ts}: last-update timestamp map
Input: λ : decay factor

```

1: function EFFECTIVEAFS( $p, t$ )
2:   return AFS[ $p$ ]  $\cdot \lambda^{t - \text{AFS}_{ts}[p]}$        $\triangleright$  Continuous decay
3: function ONPICK( $p, t$ )
4:   AFS[ $p$ ]  $\leftarrow$  EFFECTIVEAFS( $p, t$ ) + 1       $\triangleright$  Refresh, then increment
5:   AFSts[ $p$ ]  $\leftarrow t$ 
6:  $C \leftarrow \text{MIGRATOR.GETSCHEDULABLEAGGREGATORS}$   $\triangleright$  Step 1: set pool
7:  $t \leftarrow \text{Now}$ 
8:  $Q_{\max} \leftarrow \max_{p \in C} M[p].q$        $\triangleright$  Step 2: rank by queue
9:  $\mathcal{T} \leftarrow \{p \in C : M[p].q = Q_{\max}\}$        $\triangleright$  Ties
10:  $p^* \leftarrow \arg \min_{p \in \mathcal{T}} \text{EFFECTIVEAFS}(p, t)$        $\triangleright$  Break with AFS
11: ONPICK( $p^*, t$ )       $\triangleright$  Step 3: update AFS
12: schedule on  $p^*$ 

```

Figure 6: Algorithm #1 – Scheduling-pseudo-code – algorithm version of Fig. 5.

Algorithm 1 ScalingAlgorithm

```

// Collect Inputs
1:  $num\_clients \leftarrow \text{GETNUMCLIENTS}$ 
2:  $current\_replicas \leftarrow \text{GETCURRENTREPLICAS}$ 
3:  $scale\_threshold, cooldown\_time \leftarrow \text{GETFROMPERFORMANCE-MANAGER}$ 
4:  $util\_threshold \leftarrow \text{Env.UTIL\_THRESHOLD}$ 

// Step 1: Client-Based Policy
5:  $desired\_replicas \leftarrow \lceil num\_clients / scale\_threshold \rceil$ 
6: if  $desired\_replicas > current\_replicas$  then
7:   Scale out to  $desired\_replicas$ 
8:   BEGINCOOLDOWN( $cooldown\_time$ )
9: else if  $desired\_replicas < current\_replicas$  then
10:  Scale in to  $desired\_replicas$ 
11:  BEGINCOOLDOWN( $cooldown\_time$ )
12: else

// Step 2: Utilization-Based Policy (fallback)
13:  $current\_util \leftarrow training\_time / (training\_time + avg\_wait\_time + avg\_agg\_time)$ 
14: if  $current\_util < util\_threshold$  then
15:    $\lambda_{updates} \leftarrow \text{GETUPDATERATE}$ 
16:    $desired\_replicas \leftarrow \lceil \lambda_{updates} / (scale\_threshold \cdot util\_threshold) \rceil$ 
17:   if  $desired\_replicas > current\_replicas$  then
18:     Scale out to  $desired\_replicas$ 
19:     BEGINCOOLDOWN( $cooldown\_time$ )
20:   else if  $desired\_replicas < current\_replicas$  then
21:     Scale in to  $desired\_replicas$ 
22:     BEGINCOOLDOWN( $cooldown\_time$ )

```

Figure 7: Algorithm 2: Scaling Algorithm.

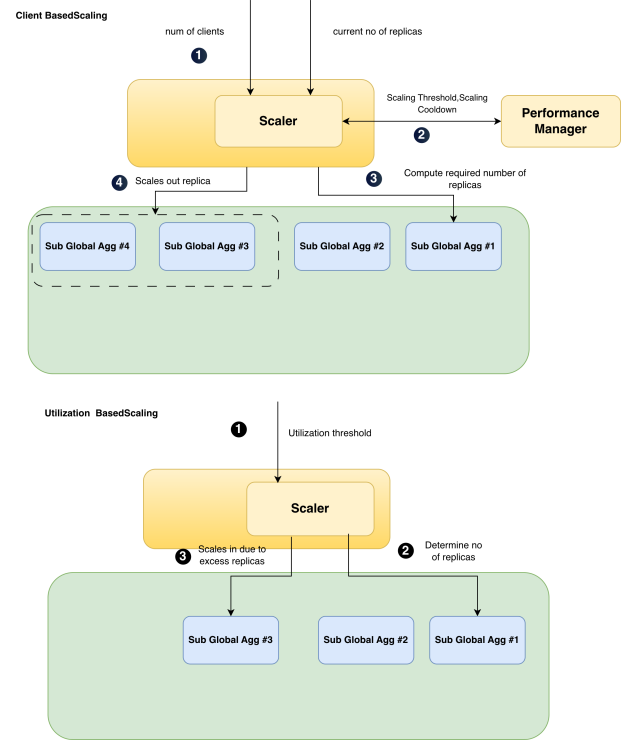


Figure 8: Scaling Algorithm

4.3 Proactive Fault Tolerance

Spot instances can be reclaimed by the cloud provider at any time, making reactive recovery insufficient. SPOTLIGHT utilizes proactive fault tolerance (FT) techniques to mitigate the impact of spot preemptions on training progress.

Migrator is responsible for two key FT operations in SPOTLIGHT: workload migration and checkpoint triggering. The Migrator derives a Kaplan-Meier model [11] from publicly available lifetime traces [28] to estimate the likelihood of aggregator failure based on aggregator age. The Migrator tracks the age and preemption events of aggregators and operates in a closed loop with the Scheduler described in Section 4.2. The Migrator proactively detects aggregators at risk of failure and removes them from the candidate pool. The Scheduler then uses the updated pool to schedule incoming client updates on the available aggregators. When migration is not possible due to aggregator unavailability, the Migrator detects this and directs the Scheduler to trigger a Failover Edge Aggregator, which acts as a temporary bridge until a replacement aggregator is provisioned. The migration mechanism is shown in Fig. 9a and b.

Checkpointing in SPOTLIGHT is triggered by the Migrator when an aggregator is identified as likely to be preempted. The aggregator persists its model state, in-buffer client contributions, and server-side optimizer state to the database. Optimizer state is saved conditionally based on the aggregation algorithm in use: stateless algorithms such as FedAvg require

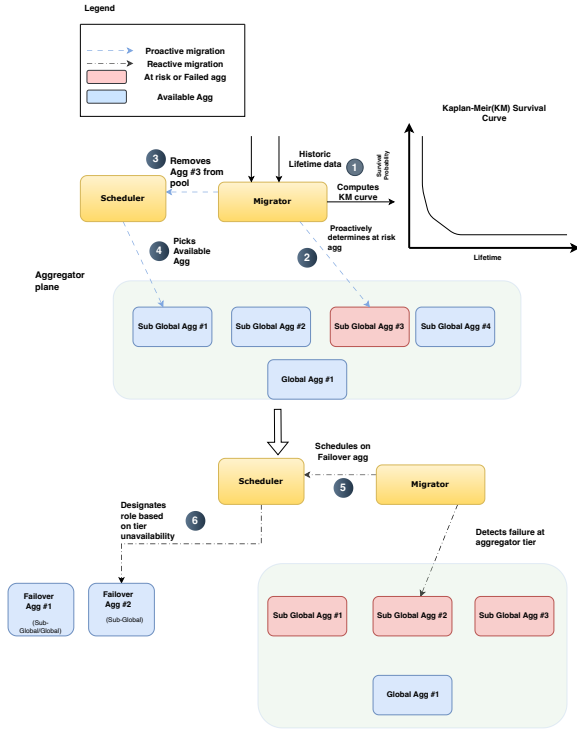


Figure 9: Migrator decision flow. The Migrator uses a Kaplan-Meier survival model [11] to compute survival probability from historic aggregator lifetime data, proactively identifying at-risk aggregators and removing them from the Scheduler’s candidate pool before failure occurs (top). When an aggregator tier fails and no aggregator in the pool is available, the Migrator detects the failure and directs the Scheduler to designate a Failover Aggregator based on which tier became unavailable (bottom).

no optimizer state to be saved, while stateful algorithms such as FedAdam persist their server-side optimizer state alongside the model. This design reduces I/O overhead compared to periodic checkpointing, persisting state only when the Migrator determines recovery is likely to be needed.

Leader Elector protects the RM, which serves as the control plane for SPOTLIGHT, coordinating scaling, migration, and scheduling across the entire system. A single RM instance would represent a critical point of failure; the RM therefore scales horizontally, introducing the need for consensus to prevent conflicting control plane decisions across instances. SPOTLIGHT uses the Raft consensus protocol [20] to maintain a single elected leader among RM instances at all times. The leader holds exclusive authority over scaling and migration decisions, pushing them to all follower instances, which use them as the basis for scheduling incoming client

updates. When the leader fails, Raft elects a successor automatically, restoring full control plane authority without interrupting scheduling operations across instances.

4.4 Elastic Spot Provisioning

SPOTLIGHT addresses two key challenges of FL systems deployed on spot infrastructure, variable client demand and spot preemption, through the coordinated operation of the components described in Sections 4.2 and 4.3. The Scaler and Performance Manager work in a closed loop to maintain aggregation throughput as client participation fluctuates, adjusting provisioning in response to observed system performance. Concurrently, the Migrator and Scheduler operate in a closed loop to handle spot preemptions: the Migrator continuously updates the pool of available aggregators based on predicted preemption risk, while the Scheduler places incoming updates on the aggregators most likely to trigger aggregation soonest within that pool. When no aggregator is available, Failover Edge Aggregators serve as a bridge. Working concurrently, these components allow SPOTLIGHT to adapt to demand variability and infrastructure instability simultaneously, without manual intervention.

5 Implementation

5.1 SPOTLIGHT Implementation

SPOTLIGHT is implemented in Python across 19.2k lines of code, reflecting the breadth of its control plane, fault tolerance, and scheduling components, and is deployed on a Kubernetes cluster. SpotLight utilizes third-party libraries including Motor, an asynchronous MongoDB driver, gRPC for inter-component communication, NumPy for algorithm implementations, and the Kubernetes Python client for cluster management. SpotLight uses MongoDB for coordination, state management, and client lifecycle tracking. System components are deployed as StatefulSets, which provide stable network identities necessary for gRPC-based inter-component communication, and communicate exclusively over gRPC, chosen over REST for its HTTP/2-native multiplexing and bidirectional streaming, which eliminates per-request connection overhead on the critical path of persistent client-to-RM streams. All inbound client connections are routed through an Envoy gateway, selected for its native gRPC support and connection load balancing. SpotLight was hosted on Chameleon Cloud [12] during evaluation. The algorithms supported by SpotLight are listed in Table 2.

Table 2: Supported Aggregation Algorithms

Algorithm	Global Model Update
FedAvg [17]	$w^{(t+1)} = \sum_{k=1}^K \frac{n_k}{N} w_k^{(t)}$
FedBuff [19]	$w^{(t+1)} = \frac{\sum_{k \in \mathcal{B}} n_k w_k^{(t)}}{\sum_{k \in \mathcal{B}} n_k}$
FedNova [27]	$m^{(t+1)} = \beta_m m^{(t)} + (1 - \beta_m) \frac{\sum_k p_k (w^{(t)} - w_k^{(t)})/\tau_k}{\sum_k p_k/\tau_k}, \quad w^{(t+1)} = w^{(t)} - \eta_k m^{(t+1)}$
FedAdam [22]	$w^{(t+1)} = w^{(t)} - \eta \cdot \frac{\hat{m}^{(t)}}{\sqrt{\hat{v}^{(t)} + \epsilon}}, \quad g = w^{(t)} - \tilde{w}$
FedYogi [22]	$w^{(t+1)} = w^{(t)} - \eta \cdot \frac{\hat{m}^{(t)}}{\sqrt{\hat{v}^{(t)} + \epsilon}}, \quad g = \tilde{w} - w^{(t)}$

5.2 Spot Emulator

A key feature of spot instances is their variability in pre-emption behavior, driven by factors including region, availability zone, instance type, temporal market conditions and cloud providers [5, 14, 23]. This variability across configurations makes real-world evaluation of application behavior on spot instances difficult to reproduce consistently and cost-prohibitive at scale, as comprehensive coverage across regions and instance types requires extensive testing for long periods of time. Therefore, to evaluate SpotLight under realistic spot conditions, we developed the Spot Emulator, a high-fidelity trace-driven emulator that replays spot VM behavior using publicly available spot availability traces [13, 28]. The emulator is designed around two key goals. First, high fidelity: the emulator replays preemption events derived from real-world availability traces, ensuring emulated behavior reflects realistic spot market conditions. Second, controller-agnostic design: the modular architecture of the Spot Emulator allows it to support Spot VM behavior replay with different cloud services providers and deploy on all platforms with minimal changes to the emulator.

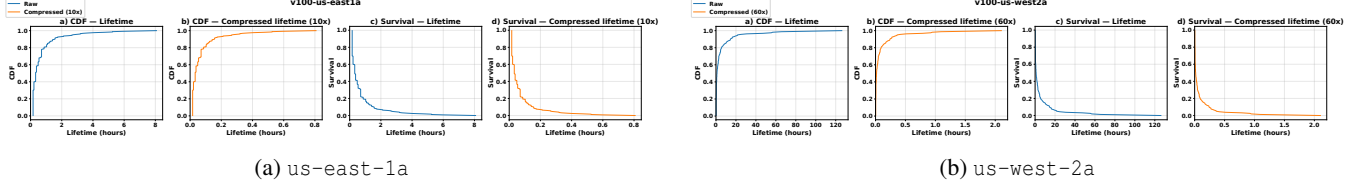


Figure 10: Lifetime and survival distributions for V100 spot instances, derived from availability traces in [28]. Panels (a) and (b) in each subfigure show the CDF of instance lifetime in raw and time-compressed form; panels (c) and (d) show the corresponding Kaplan-Meier survival curve. The time compression factor ($10\times$ for us-east-1a, $60\times$ for us-west-2a) is a configurable parameter of the Spot Emulator, set independently per region to fit evaluation run-time constraints; it is not derived from the trace data itself. The Spot Emulator samples instance lifetimes from these distributions to replay preemption events during evaluation.

The lifetime distribution used to replay spot behavior is derived from the availability traces in [28]. The availability trace is used to derive a CDF, which allows the emulator to reliably replay spot instance lifetime dynamics. To ensure that evaluation cost is realistic, price traces obtained from [13] are used to generate realistic execution costs. As shown in Figure 10, survival probability decreases monotonically with instance lifetime, reflecting the inherent unreliability of spot instances over time.

The Spot Emulator consists of five components. The first is the *Distribution Extractor*, which filters the availability trace by region, availability zone, and OS, and derives the lifetime CDF from the filtered data. The second is the *Lifecycle Manager*, which samples a lifetime for each newly provisioned VM from the CDF within a configured range, excluding tail outliers that represent atypically short-lived or long-lived instances, and maintains an internal clock tracking the lifetime of all active spot VMs. The third component is the *Executor*, which terminates instances via the Kubernetes API once their assigned lifetime expires. The fourth component is the *Control Plane*, which manages the execution order of the other components and exposes configuration parameters to the user, including the sampling range and time compression factor. The fifth component is the *Cost Report Generator*, which uses price traces from [13] to produce accurate execution cost reports.

Execution begins when the Control Plane is configured with the lifetime trace source, sampling range, and time compression factor. The Control Plane invokes the Distribution Extractor to generate the lifetime CDF, which is passed to the Lifecycle Manager. The Lifecycle Manager samples a lifetime from the CDF within the configured range, assigns it to each pod, and tracks elapsed time via an internal clock. When a pod’s assigned lifetime elapses, the Lifecycle Manager invokes the Executor to terminate the pod.

The Spot Emulator’s logical model maps directly onto a Kubernetes cluster. The Cluster represents the datacenter, and the pods scheduled onto its worker nodes represent the Spot VMs, each assigned a lifetime and terminated once that lifetime

expires. SpotLight is deployed on top of this emulated cluster, allowing it to be evaluated under realistic spot preemption behavior.

References

- [1] Amazon Web Services. Spot Instance interruption notices. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html>.
- [2] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2022.
- [3] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards Federated Learning at Scale: System Design. In *The Second Conference on Machine Learning and Systems (SysML)*, 2019.
- [4] DDPS Lab. SpotLake. <https://spotlake.ddps.cloud/>, 2026.
- [5] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. Parcae: proactive, liveput-optimized dnn training on preemptible instances. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*, NSDI'24, USA, 2024. USENIX Association.
- [6] Jianwei Hao, Ting Jiang, Wei Wang, and In Kee Kim. An Empirical Analysis of VM Startup Times in Public IaaS Clouds. In *IEEE International Conference on Cloud Computing (CLOUD)*, 2021.
- [7] Dzmitry Huba, John Nguyen, Kshitiz Malik, Ruiyu Zhu, Mike Rabbat, Ashkan Yousefpour, Carole-Jean Wu, Hongyuan Zhan, Pavel Ustinov, Harish Srinivas, Kaikai Wang, Anthony Shoumikhin, Jesik Min, and Mani Malek. PAPAYA: Practical, private, and scalable federated learning. In *Proceedings of the 5th Conference on Machine Learning and Systems (MLSys 2022)*, 2022.
- [8] David Irwin, Prashant J. Shenoy, Pradeep Ambati, Praateek Sharma, Supreeth Shastri, and Ahmed Ali-Eldin. The Price Is (Not) Right: Reflections on Pricing for Transient Cloud Servers. In *28th International Conference on Computer Communication and Networks (ICCCN)*, 2019.
- [9] K. R. Jayaram, Vinod Muthusamy, Gegi Thomas, Ashish Verma, and Mark Purcell. Adaptive aggregation for federated learning. In *IEEE International Conference on Big Data (Big Data)*, 2022.
- [10] Pedro Joaquim, Manuel Bravo, Luís Rodrigues, and Miguel Matos. Hourglass: Leveraging transient resources for time-constrained graph processing in the cloud. In *The Fourteenth EuroSys Conference (EuroSys)*, 2019.
- [11] E. L. Kaplan and Paul Meier. Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282):457–481, 1958.
- [12] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.
- [13] Sungjae Lee, Jaeil Hwang, and Kyungyong Lee. SpotLake: Diverse Spot Instance Dataset Archive Service. In *IEEE International Symposium on Workload Characterization (IISWC)*, pages 242–255, 2022.
- [14] Zhifei Li, Tian Xia, Ziming Mao, Zihan Zhou, Ethan J. Jackson, Jamison Kerney, Zhanghao Wu, Pratik Mishra, Yi Xu, Yifan Qiao, Scott Shenker, and Ion Stoica. SkyNomad: On using multi-region spot instances to minimize AI batch job cost. *CoRR*, abs/2601.06520, 2026.
- [15] Yang Liu, Tao Fan, Tianjian Chen, Qian Xu, and Qiang Yang. FATE: An industrial grade platform for collaborative learning with data protection. *Journal of Machine Learning Research*, 22(226):1–6, 2021.
- [16] Heiko Ludwig, Nathalie Baracaldo, Gegi Thomas, Yi Zhou, Ali Anwar, Shashank Rajamoni, Yuya Ong, Jayaram Radhakrishnan, Ashish Verma, Mathieu Sinn, et al. IBM federated learning: An enterprise framework white paper V0.1. *arXiv preprint arXiv:2007.10987*, 2020.
- [17] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.
- [18] Microsoft. Build workloads on spot virtual machines. <https://learn.microsoft.com/en-us/azure/architecture/guide/spot/spot-eviction>.
- [19] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. Federated learning with buffered asynchronous

- aggregation. In *International Conference on Artificial Intelligence and Statistics*, pages 3581–3607. PMLR, 2022.
- [20] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, Philadelphia, PA, June 2014. USENIX Association.
- [21] Shixiong Qi, K. K. Ramakrishnan, and Myungjin Lee. LIFL: A Lightweight, Event-driven Serverless Platform for Federated Learning. In *The Seventh Annual Conference on Machine Learning and Systems (MLSys)*, 2024.
- [22] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. Adaptive federated optimization. In *International Conference on Learning Representations (ICLR)*, 2021.
- [23] Prateek Sharma, JCS Kadupitiya, and Vikram Jadhao. Modeling constrained preemption dynamics of transient cloud servers. *arXiv*, pages arXiv–1911, 2019.
- [24] Supreeth Subramanya, Amr Rizk, and David Irwin. Cloud Spot Markets are Not Sustainable: The Case for Transient Guarantees. In *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2016.
- [25] Yan Sun, Li Shen, Hao Sun, Liang Ding, and Dacheng Tao. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(12):14453–14464, 2023.
- [26] Ewen Wang, Boyi Chen, Mosharaf Chowdhury, Ajay Kannan, and Franco Liang. FLINT: A Platform for Federated Learning Integration. In *The Sixth Conference on Machine Learning and Systems (MLSys)*, 2023.
- [27] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. Tackling the objective inconsistency problem in heterogeneous federated optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [28] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. Can’t Be Late: Optimizing Spot Instance Savings under Deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.
- [29] Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liquan Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Íñigo Goiri, Eli Cortez, Terry Yang, Victor Rühle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. Snape: Reliable and low-cost computing with mixture of spot and on-demand vms. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.